

# LAB 1

## DevOps Foundations & Continuous Integration

### Task 7 - Create a Jenkinsfile Defining Pipeline Stages

Student: **Kunal**

#### Objective

Create a Jenkinsfile defining the required pipeline stages **Build**, **Test**, and **Deploy**, and demonstrate a declarative Jenkins pipeline execution.

#### Implementation Summary

A declarative Jenkinsfile was created in the Git repository and committed to the main branch. Jenkins was configured to load the Jenkinsfile from GitHub using **Pipeline script from SCM**. The resulting pipeline execution demonstrates all three required stages and a successful final result.

| Component             | Implementation           |
|-----------------------|--------------------------|
| Pipeline type         | Declarative Pipeline     |
| Pipeline definition   | Jenkinsfile              |
| Stages                | Build → Test → Deploy    |
| Source control        | GitHub repository        |
| Branch                | main                     |
| Jenkins configuration | Pipeline script from SCM |
| Execution result      | Finished: SUCCESS        |

# 1. Jenkinsfile Definition

The following declarative structure was used. It directly satisfies the Task 7 requirement for Build, Test, and Deploy stages.

```
pipeline {
    agent any

    stages {
        stage('Build') {
            steps {
                echo 'Build stage completed successfully.'
            }
        }

        stage('Test') {
            steps {
                echo 'Test stage completed successfully.'
            }
        }

        stage('Deploy') {
            steps {
                echo 'Deploy stage completed successfully.'
            }
        }
    }

    post {
        success {
            echo 'Pipeline completed successfully.'
        }
    }
}
```

## Stage purpose

**Build:** represents the application build phase. **Test:** represents validation/testing. **Deploy:** represents deployment. The **post { success }** section reports successful completion after the stages finish.

# 1. Jenkinsfile source code

The terminal output shows the declarative **Jenkinsfile** containing three required pipeline stages: **Build**, **Test**, and **Deploy**. A successful post-build action is also defined.

```
kunal_prajapati@LAPTOP-45509TK7:~/lab1-devops$ cat Jenkinsfile
pipeline {
    agent any

    stages {
        stage('Build') {
            steps {
                echo 'Build stage completed successfully.'
            }
        }

        stage('Test') {
            steps {
                echo 'Test stage completed successfully.'
            }
        }

        stage('Deploy') {
            steps {
                echo 'Deploy stage completed successfully.'
            }
        }
    }

    post {
        success {
            echo 'Pipeline completed successfully.'
        }
    }
}
```

```
kunal_prajapati@LAPTOP-45509TK7:~/lab1-devops$ |
```

Figure 1. Jenkinsfile source code.

## 2. Jenkinsfile editing step

The terminal evidence shows the Jenkinsfile being opened from the **~/lab1-devops** repository for configuration and editing.

```
kunal_prajapati@LAPTOP-45509TK7:~/lab1-devops$ cd ~/lab1-devops  
kunal_prajapati@LAPTOP-45509TK7:~/lab1-devops$ nano Jenkinsfile
```

Figure 2. Jenkinsfile editing step.

### 3. Jenkinsfile committed and pushed to GitHub

The Git output confirms the Jenkinsfile change was pushed to the GitHub repository. The **main** branch is updated successfully.

```
kunal_prajapati@LAPTOP-45509TK7:~/lab1-devops$ git push
Username for 'https://github.com': kunalprajapati14042005
Password for 'https://kunalprajapati14042005@github.com':
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 8 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 443 bytes | 147.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0
To https://github.com/kunalprajapati14042005/lab1-devops-version-control.git
 f7e9d4f..84e1e12  main -> main
kunal_prajapati@LAPTOP-45509TK7:~/lab1-devops$ |
```

Figure 3. Jenkinsfile committed and pushed to GitHub.

## 4. Repository working-tree verification

Git status confirms the local repository is on **main**, synchronized with **origin/main**, with a clean working tree.

A terminal window with a black background and white text. The prompt is 'kunal\_prajapati@LAPTOP-45509TK7:~/lab1-devops\$'. The command 'git status' has been executed, and the output is displayed in several lines: 'On branch main', 'Your branch is up to date with 'origin/main'.', and 'nothing to commit, working tree clean'.

```
+7e9d4f..84e1e12  main -> main
kunal_prajapati@LAPTOP-45509TK7:~/lab1-devops$ git status
On branch main
Your branch is up to date with 'origin/main'.
nothing to commit, working tree clean
```

Figure 4. Repository working-tree verification.

## 5. Jenkins Pipeline configured from SCM

Jenkins is configured with **Pipeline script from SCM**, using Git as the SCM and the specified GitHub repository.

### Pipeline

Define your Pipeline using Groovy directly or pull it from source control.

#### Definition

Pipeline script from SCM

#### SCM ?

Git

#### Repositories ?

##### Repository URL ?

https://github.com/kunalprajapati14042005/lab1-devops-version-control.git

##### Credentials ?

- none -

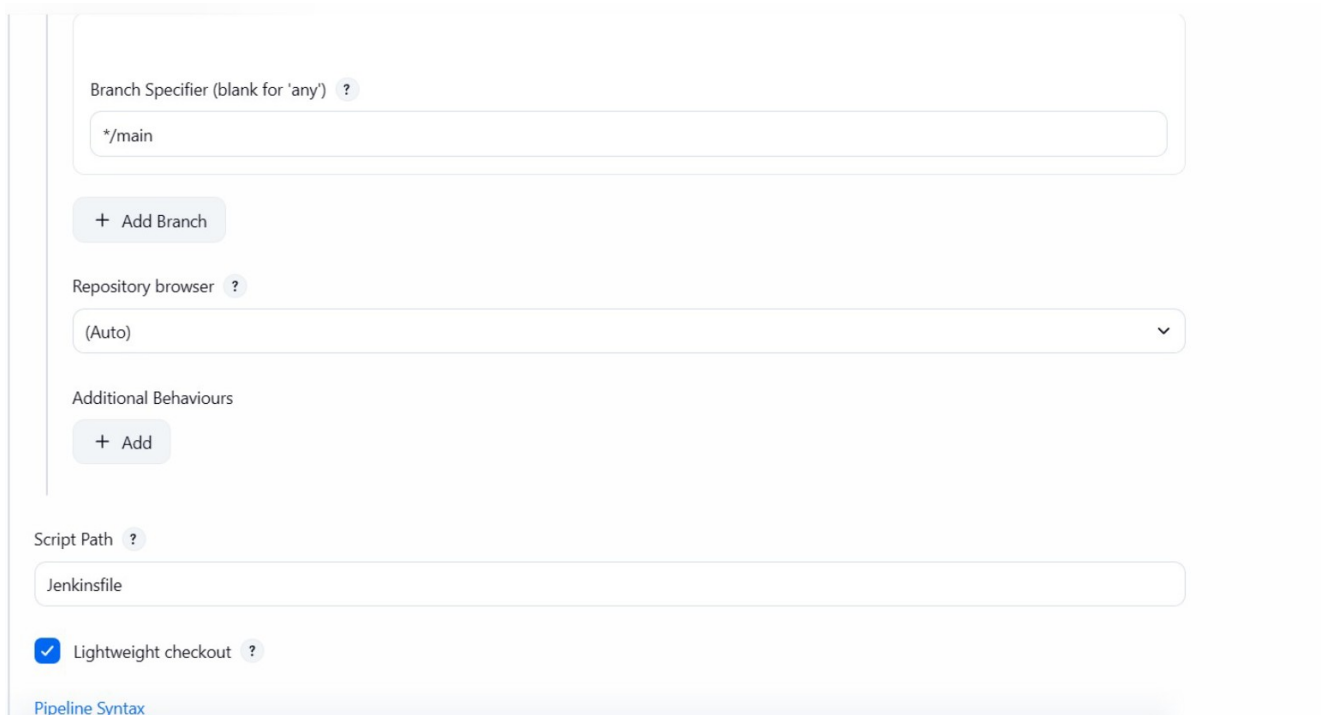
+ Add

Advanced

Figure 5. Jenkins Pipeline configured from SCM.

## 6. Branch and Jenkinsfile path configuration

The Pipeline configuration specifies branch **\*/main** and script path **Jenkinsfile**. Lightweight checkout is enabled.



The screenshot displays the Jenkins Pipeline configuration interface. It features a sidebar on the left with a vertical line. The main configuration area includes the following elements:

- Branch Specifier (blank for 'any') ?**: A text input field containing the value `*/main`.
- + Add Branch**: A button to add additional branch specifiers.
- Repository browser ?**: A dropdown menu currently set to `(Auto)`.
- Additional Behaviours**: A section with a **+ Add** button.
- Script Path ?**: A text input field containing the value `Jenkinsfile`.
- Lightweight checkout ?**: A checkbox that is checked, indicating that lightweight checkout is enabled.

At the bottom left of the configuration area, there is a link labeled [Pipeline Syntax](#).

Figure 6. Branch and Jenkinsfile path configuration.



## 7. Jenkins pipeline checkout execution

Jenkins retrieves the Jenkinsfile/source from the GitHub repository and checks out the main branch. This demonstrates SCM-based pipeline execution.

Console

Download

Copy

View as plain text

```
Started by user Kunal Prajapati
Obtained Jenkinsfile from git https://github.com/kunalprajapati14042005/lab1-devops-version-control.git
[Pipeline] Start of Pipeline
[Pipeline] node
Running on Jenkins in /var/lib/jenkins/workspace/Kunal-Lab1-Basic-CICD
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Declarative: Checkout SCM)
[Pipeline] checkout
Selected Git installation does not exist. Using Default
The recommended git tool is: NONE
No credentials specified
> git rev-parse --resolve-git-dir /var/lib/jenkins/workspace/Kunal-Lab1-Basic-CICD/.git # timeout=10
Fetching changes from the remote Git repository
> git config remote.origin.url https://github.com/kunalprajapati14042005/lab1-devops-version-control.git # timeout=10
Fetching upstream changes from https://github.com/kunalprajapati14042005/lab1-devops-version-control.git
> git --version # timeout=10
> git --version # 'git version 2.43.0'
> git fetch --tags --force --progress -- https://github.com/kunalprajapati14042005/lab1-devops-version-control.git
+refs/heads/*:refs/remotes/origin/* # timeout=10
> git rev-parse refs/remotes/origin/main^{commit} # timeout=10
Checking out Revision 84e1e12b2fc4f0c8d402f05fa83b86ba5617b1b8 (refs/remotes/origin/main)
> git config core.sparsecheckout # timeout=10
> git checkout -f 84e1e12b2fc4f0c8d402f05fa83b86ba5617b1b8 # timeout=10
Commit message: "Add Jenkins CI/CD pipeline"
```

Figure 7. Jenkins pipeline checkout execution.

## 8. Build, Test and Deploy stages executed

The Jenkins console shows successful execution of the **Build**, **Test**, and **Deploy** stages.

```
#2 ▾ 🔍 ⚙️

> git rev-list --no-walk f7e9d4ff18296fb9d3bb079e6ccc38cc0d0fe7d1 # timeout=10
[Pipeline] }
[Pipeline] // stage
[Pipeline] withEnv
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Build)
[Pipeline] echo
Build stage completed successfully.
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Test)
[Pipeline] echo
Test stage completed successfully.
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Deploy)
[Pipeline] echo
Deploy stage completed successfully.
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Declarative: Post Actions)
[Pipeline] echo
Pipeline completed successfully.
[Pipeline] }
```

Figure 8. Build, Test and Deploy stages executed.

## 9. Pipeline completion

The console confirms the post-build action and ends with **Finished: SUCCESS**, demonstrating successful pipeline execution.

```
[Pipeline] { (Test)
[Pipeline] echo
Test stage completed successfully.
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Deploy)
[Pipeline] echo
Deploy stage completed successfully.
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Declarative: Post Actions)
[Pipeline] echo
Pipeline completed successfully.
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

Figure 9. Pipeline completion.

## 10. Jenkins Stage View

The Jenkins Stage View provides visual evidence that the pipeline stages completed successfully. Build #2 shows all required stages in green.

### Stages

September 9, 2026

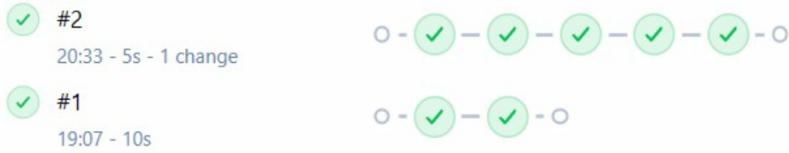


Figure 10. Jenkins Stage View.

## 12. Requirement Mapping

| Task 7 requirement               | Evidence                                                                                     |
|----------------------------------|----------------------------------------------------------------------------------------------|
| Create a Jenkinsfile             | Jenkinsfile source is shown in the terminal evidence.                                        |
| Define Build stage               | Jenkinsfile contains stage('Build'); execution evidence shows Build completed successfully   |
| Define Test stage                | Jenkinsfile contains stage('Test'); execution evidence shows Test completed successfully     |
| Define Deploy stage              | Jenkinsfile contains stage('Deploy'); execution evidence shows Deploy completed successfully |
| Declarative or scripted pipeline | The Jenkinsfile uses declarative pipeline syntax beginning with pipeline { and agent any     |
| Demonstrate pipeline execution   | Jenkins is configured with Pipeline script from SCM and loads the Jenkinsfile from GitHub    |
| Successful execution             | Console output ends with Finished: SUCCESS and Stage View shows successful stage             |

## Conclusion

Task 7 was successfully implemented and demonstrated. The Jenkinsfile defines the required Build, Test, and Deploy stages using declarative syntax. Jenkins loads the Jenkinsfile from the GitHub repository through SCM, executes the pipeline, and reports **Finished: SUCCESS**. The Stage View further confirms successful completion of the pipeline stages.